SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Mateo Zović

VIŠESTRUKO POTPISIVANJE NOVČANIKA

PROJEKTNI ZADATAK

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Mateo Zović

Studij: Organizacija poslovnih sustava

VIŠESTRUKO POTPISIVANJE NOVČANIKA

PROJEKTNI ZADATAK

Mentori:

Prof. dr. sc. Neven Vrček

Snježana Križanić, mag. inf.

Varaždin, travanj 2025.

Sažetak

U radu je obrađena tema višestrukog potpisivanja (eng. *multisig*) u kontekstu digitalnih novčanika i pametnih ugovora. Višestruko potpisivanje omogućuje sigurnije upravljanje sredstvima jer zahtijeva odobrenje više korisnika prije izvršenja transakcije. Na taj način smanjuje se rizik od krađe, pogrešaka ili neovlaštenog pristupa. Ovaj pristup posebno je koristan kada novčanik koristi više osoba, poput članova obitelji, timova ili organizacija. Prvi dio rada obrađuje teorijsku podlogu višestrukog potpisivanja, njegovih prednosti i primjene u stvarnom svijetu. U drugom dijelu, praktičnom, izrađen je pametni ugovor pomoću programskog jezika Solidity, a razvoj je proveden u online alatu Remix - Ethereum IDE. Izrađeni pametni ugovor omogućuje unos prijedloga transakcije, odobravanje i izvršavanje nakon što ju potvrdi dovoljan broj korisnika. Naposljetku se testiranjem ugovora provjerava funkcionalnost i sigurnost samog ugovora te se testiraju slučajevi pogrešaka. Rezultati pokazuju da ovakav sustav pruža visoku razinu zaštite i transparentnosti, što je od velike važnosti u suvremenom digitalnom okruženju gdje je sigurnost ključno pitanje.

Ključne riječi: višestruko potpisivanje; digitalni novčanik; transakcije; pametni ugovor; Solidity; sigurnost; transparentnost;

Sadržaj

1. Uvod 1

2. Općenito o blockchain-u i pametnim ugovorima 2

3. Višestruko potpisivanje 3

3.1. Povijest 3

3.2. Primjena 4

3.3. Prednosti 4

4. Razvojna okolina pametnog ugovora 6

4.1. Osnove Remix Online IDE okruženja 6

4.2. Osnove Solidity jezika 7

5. Opis slučaja korištenja 9

6. Implementacija pametnog ugovora 10

6.1. Inicijalizacija varijabli 10

6.2. Konstruktor 11

6.3. Predlaganje transakcije 11

6.4. Odobravanje transakcije 12

6.5. Izvršenje transakcije 12

7. Testiranje pametnog ugovora 14

7.1. Pokretanje digitalnog novčanika (Deploy) 14

7.2. Provedba transakcije 15

7.3. Simulacija pogrešaka i lažnih unosa 17

8. Zaključak 18

Popis literature 19

Popis slika 20

1. Uvod

U posljednjih desetak godina digitalne valute poput Bitcoina i Ethereuma sve više dobivaju na važnosti i mijenjaju način na koji ljudi razmišljaju o novcu, plaćanju i upravljanju imovinom. S razvojem tehnologije blockchain, koja stoji u pozadini ovih valuta, pojavili su se novi načini za pohranu i razmjenu vrijednosti bez potrebe za posrednicima poput banaka. Međutim, s tim novim mogućnostima dolaze i novi izazovi, posebice kada je riječ o sigurnosti i zaštiti digitalne imovine.

Jedno od rješenja za povećanje sigurnosti digitalnih novčanika i transakcija je koncept višestrukog potpisivanja (eng. *multi-signature*, skraćeno *multisig*). Radi se o jednostavnoj, ali vrlo učinkovitoj ideji, umjesto da jedna osoba ima potpunu kontrolu nad novčanikom, sustav zahtijeva da više osoba potvrdi neku transakciju prije nego što se ona može izvršiti. Na taj način se smanjuje mogućnost prijevare, krađe ili pogreške, jer odluka mora proći kroz više provjera. Višestruko potpisivanje se danas koristi u različitim područjima, kod zajedničkih novčanika koje koristi više članova tima ili organizacije, kod obiteljskog upravljanja sredstvima, kao i u raznim decentraliziranim financijskim aplikacijama. Osim povećane sigurnosti, ovaj pristup donosi veću odgovornost i transparentnost među korisnicima.

U nastavku će biti prikazano što je višestruko potpisivanje, zašto se koristi i koje probleme rješava. Na jednostavan način prikazat će se višestruko potpisivanje u teoriji i u praksi, kroz programiranje pametnog ugovora u Remix okruženju. Cilj je, kroz praktični dio, predstaviti primjenu višestrukog potpisivanja novčanika u stvarnom životu, kako bi se ukazao značaj ove tehnologije u današnjem digitalnom svijetu.

2. Općenito o blockchain-u i pametnim ugovorima

Blockchain (lanac blokova) je tehnologija koja je došla do velikog izražaja pojavom prvih kripto valuta. Tako primjerice, „Bitcoin blockchain kombinira postojeću Blockchain tehnologiju s tehnologijom šifriranja kako bi služio kao platforma za digitalnu valutu. (…) Blockchain je baza podataka: distribuirana, decentralizirana baza podataka.“[1] Baza podataka je pojam koji označava da se radi s podacima, bilo spremanje, uređivanje ili dohvaćanje. Distribuirana označava da su podaci te baze podijeljeni na više računala koja su povezana putem interneta, a nazivaju se čvorovi. Blockchain tehnologija omogućuje sigurno pohranjivanje i razmjenu podataka bez potrebe za središnjim posrednikom poput banke. Riječ je o distribuiranoj bazi podataka koja bilježi sve transakcije u lancu blokova (blockchain), a svaki novi blok sadrži podatke o prethodnim blokovima. Jednom upisana informacija u blockchain ne može se mijenjati bez pristanka većine sudionika u mreži. Ovime se osigurava visoka razina transparentnosti i sigurnosti, što sustav čini izuzetno otpornim na manipulacije i prijevare.

Pametni ugovori (eng. *smart contracts*) predstavljaju programske kodove koji se izvršavaju na blockchain-u kada su ispunjeni unaprijed definirani uvjeti. „Pametni ugovor mali je računalni program koji je pohranjen na Blockchainu zajedno s podacima o transakciji.“[2] Za razliku od tradicionalnih ugovora, koji ovise o trećim stranama za izvršenje, pametni ugovori automatski izvršavaju unaprijed definirane radnje. Time se štedi vrijeme, smanjuju troškovi i povećava sigurnost pametnih ugovora. „Osnovna funkcija pametnog ugovora je izvršavanje blockchain transakcije kada je ispunjen uvjet: AKO adresa primi novčić, ONDA prenijeti 10% na alternativnu adresu.“[2] Najčešće se koriste na blockchain platformi Ethereum, gdje programeri pišu pametne ugovore u programskom jeziku Solidity. Njihova primjena je raznolika, od automatizacije plaćanja, upravljanja imovinom i glasanja, do izgradnje složenijih decentraliziranih aplikacija (DApps). Pametni ugovori i blockchain zajedno omogućuju stvaranje sigurnog, decentraliziranog digitalnog ekosustava.

3. Višestruko potpisivanje

Pojam višestrukog potpisivanja, kao što sam naziv već govori, podrazumijeva odobravanje neke transakcije, događaja ili slične aktivnosti putem više potpisa. Prema tome, potpis mora doći od više osoba, kako bi se određeni događaj, odnosno transakcija izvršila. Ova metoda se koristi za postizanje veće sigurnosti i transparentnosti. „Tehnologija višestrukog potpisivanja koristi se u digitalnim novčanicima kako bi se povećala sigurnost kriptografskih transakcija. Zahtijeva odobrenje više strana prije nego što se transakcija može obraditi, čime se smanjuje rizik od prijevare ili krađe. Tehnologija višestrukog potpisivanja posebno je korisna za transakcije velikih razmjera, gdje može sudjelovati više strana.“ [3] Višestruko potpisivanje povećava sigurnost digitalnih transakcija zahtijevajući odobrenje više strana, čime smanjuje rizik od prijevare i krađe. Ova tehnologija posebno je korisna za transakcije velikih razmjera, kriptoburze i decentralizirane organizacije, gdje osigurava veću transparentnost i zaštitu imovine.

3.1. Povijest

Višestruko potpisivanje se temelji na kriptiranju poruke, odnosno u ovom slučaju, kriptiranju potpisa. Teškim matematičkim formulama i izračunima se nastoji sakriti sadržaj poruke, a pomoću koncepta privatnog i javnog ključa se lako može provjeriti ispravnost određene poruke. Samim time, višestruko potpisivanje se pojavljuje pojavom moderne kriptografije, odnosno pojavom RSA kriptosustava. „… 1977. godine izumili prvi kriptosustav s javnim ključem koji su nazvali RSA kriptosustav. Sigurnost RSA kriptosustava leži u činjenici da je faktorizacija velikih prirodnih brojeva na produkt dva prosta broja izuzetno teška, …“[4] Pojam višestrukog potpisivanja je tek zaživio pojavom Bitcoina. „Otac kriptografije smatra se Japanac Satoshi Nakamoto koji je početkom 2009. god. pustio u uporabu prvi puta Bitcoin. Bitcoin je bio zamišljen kao sigurna valuta, a za njegovu sigurnost bi se brinuli jedinstveni potpisi koji bi bili ugrađeni u softver kroz sistem slagalice. Bitcoin je prvi oblik digitalnog novca koji za prijenos vrijednosti koristi kriptografske algoritme.“[5]. Iako u početku Bitcoin nije koristio digitalne potpise za autentifikaciju digitalnih transakcija, naglim porastom vrijednosti valute Bitcoina morala se uvesti određena doza sigurnosti transakcija. Prema tome uvedena je provjera digitalnog potpisivanja, gdje pojedinac koristi svoj kriptografski privatni ključ za potpisivanje transakcije. Ova procedura daje dodatni sloj sigurnosti te se prema njoj temelji i metoda višestrukog potpisivanja koja uključuje digitalne potpise više pojedinaca.

3.2. Primjena

Danas višestruko potpisivanje ima široku primjenu, a najčešće se odnosi na pametne ugovore, blockchain (lanac blokova) i kriptovalute. Sve navedene stavke su međusobno povezane i njima se ostvaruju digitalne transakcije. Digitalne transakcije uključuju prijenos podataka, bilo informacijskih ili financijskih putem internet mreže. Kao što je već prije navedeno, glavna ideja uvođenja višestrukog potpisivanja je da sva vlast nije u rukama pojedinca već više osoba. Time se sprečavaju neovlaštene transakcije i krađe resursa određene organizacije. Navedeni resurs organizacije može se definirati kao digitalni novčanik, pa prema tome višestruko potpisivanje novčanika, podrazumijeva osiguravanje digitalnih resursa organizacije od napada i krađe. „Digitalni novčanik predstavlja oblik elektronske verzije klasičnog, fizičkog novčanika koji se može koristiti na smartphone-u, tabletu i pametnom satu, a uveden je radi povećanja sigurnosti i učinkovitosti prilikom bezgotovinskog plaćanja.“[6]. Osim sigurnosnih pitanja, omogućuje i identificiranje što organizacija uistinu posjeduje. Ukratko rečeno, primjena višestrukog potpisivanja je način upravljanja zajedničkom imovinom, kao i decentralizacija vlasti nad određenim resursom.

Potreba za višestrukim potpisivanjem se može objasniti na primjeru velike organizacije, koja želi izvesti veliku novčanu transakciju. Vlasnik organizacije nije pojedinac, već je vlast podijeljena na pet osoba. U ovom slučaju, većina vlasnika mora odobriti transakciju, odnosno tri od pet vlasnika. Ideja iza višestrukog potpisivanja je da vlast nije u rukama jednog pojedinca već da se dijeli među više osoba, čime se nastoji izbjeći neželjene situacije poput prijevara i krađa. Još jedna prednost višestrukog potpisivanja je u slučaju da netko od vlasnika izgubi svoj profil (zaboravi lozinku ili je hakiran), ostali vlasnici će i dalje moći donijeti odluku o transakciji po načelu većine. U drugom slučaju, gdje je haker dobio pristup profilu vlasnika ne može sam izvršiti određenu transakciju bez odobrenja drugih vlasnika. Na ovaj način višestruko potpisivanje jamči visoku sigurnost i pouzdanost prilikom provođenja online transakcija.

3.3. Prednosti

Prednosti koje se ostvaruju višestrukim potpisivanjem u odnosu na tradicionalne metode obuhvaćaju veću sigurnost, smanjenje rizika od prijevare i decentralizacija, bolju zaštitu od gubitka ključa i transparentnost. Obje metode se odnose na upravljanje resursima pohranjene na digitalnim novčanicima. U nastavku su opisane prednosti višestrukog potpisivanja digitalnog novčanika.

a) Sigurnost – dok se tradicionalne metode oslanjaju na jednu lozinku ili privatni ključ, višestruko potpisivanje zahtijeva više lozinki odnosno ključeva za autorizaciju transakcije, čime se povećava sigurnost.
b) Smanjenje rizika od prijevare i decentralizacija – kod tradicionalnih metoda sva vlast je u rukama pojedinačne osobe, dok se u višestrukom potpisivanju dijeli odgovornosti i vlast između više osoba, čime se smanjuje rizik od prijevare.
c) Zaštita od gubitka ključa – osoba kod tradicionalne metode gubi sva digitalna sredstva u slučaju da izgubi svoj privatni ključ, a kod metode višestrukog potpisivanja postoji mogućnost oporavka korištenjem rezervnih ključeva unutar grupe osobe koje imaju isti skup ovlasti.
d) Transparentnost – u tradicionalnim sustavima često postoji ograničeni pristup podacima o transakcijama, dok se kod višestrukog potpisivanja svaka transakcija može pratiti pomoću blokchaina, čime se ostvaruje transparentnost i samim time povjerenje.

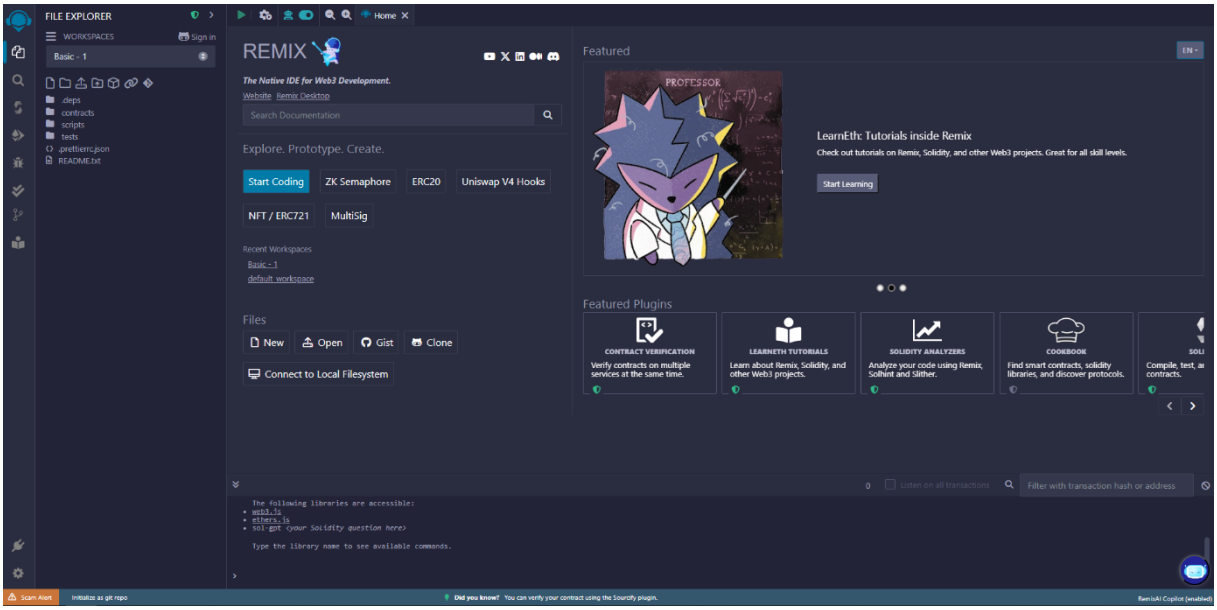
Dakle, može se zaključiti da metoda višestrukog potpisivanja nudi mnogobrojne prednosti u odnosu na tradicionalne metode. Najviše se od svega ističe povećana sigurnost, decentralizacija vlasti i transparentnost.

4. Razvojna okolina pametnog ugovora

Kao razvojna okolina za izradu projekta koristit će se Solidity jezik u Remix Online IDE okruženju. Remix IDE je online alat s grafičkim sučeljem koji služi za razvoj i izradu pametnih ugovora. „Koriste ga i stručnjaci i početnici, omogućujući brz početak rada. Remix je kompatibilan s drugim alatima i omogućuje jednostavan proces implementacije na blockchain po izboru korisnika. Poznat je po svom vizualnom debuggeru i često se koristi kao platforma za učenje Ethereuma.“[7] Ethereum je decentralizirana blockchain platforma koja omogućuje razvoj i izvršavanje pametnih ugovora i decentraliziranih aplikacija. Pokrenut je 2015. godine, a za razliku od Bitcoina, koji je primarno digitalna valuta, Ethereum omogućuje programibilne transakcije putem vlastitog programskog jezika Solidity. Njegova specifična kriptovaluta Ether se koristi za plaćanje transakcijskih naknada i izvođenje pametnih ugovora.

4.1. Osnove Remix Online IDE okruženja

Kao što je već ranije spomenuto, Remix IDE je razvojno okruženje za pisanje, testiranje i implementaciju pametnih ugovora u Solidity jeziku. Prilikom pokretanja online razvojnog okruženja, dostupnog na poveznici Remix Online IDE [8] otvara se početni zaslon kao što je prikazano na sljedećoj slici [Slika 1].



Slika 1: Remix Online IDE - Početni zaslon

Na lijevoj strani ekrana nalaze se ikone za karakteristične radnje poput ikone pristupa datotekama, traženje, kompajler, prijenos i pokretanje programa, sustav za debagiranje, testiranje, spajanje na Git i slične radnje. U prozoru do je otvorena odabrana opcija, u ovom

slučaju je to upravljanje datotekama (eng. *File explorer*). Ovdje se nalaze sve mape i datoteke koje su zadužene za izvođenje i pokretanje pametnog ugovora. U gornjem dijelu prozora su otvorene sve nedavne kartice i datoteke, te isti daj odjeljak sadrži gumbe za kompajliranje određenog programa. U sredini prozora se piše programski kod, iako je na prikazanoj slici trenutačno otvoren početni prozor koji nudi opcije za početak programiranja. Na dnu ekrana se nalazi konzola koja prikazuje greške i upozorenja izvršenog programskog koda.

4.2. Osnove Solidity jezika

Solidity je objektno orijentirani programski jezik koji se koristi za kreiranje pametnih ugovora. „Solidity je moćan programski jezik dizajniran posebno za pisanje pametnih ugovora na Ethereum blockchainu. Pomoću Solidityja, programeri mogu definirati pravila i ponašanje decentraliziranih aplikacija (DApps).“[9] Pomoću njega se programiraju sigurni i transparentni pametni ugovori koji omogućuju automatizaciju transakcija bez posrednika.

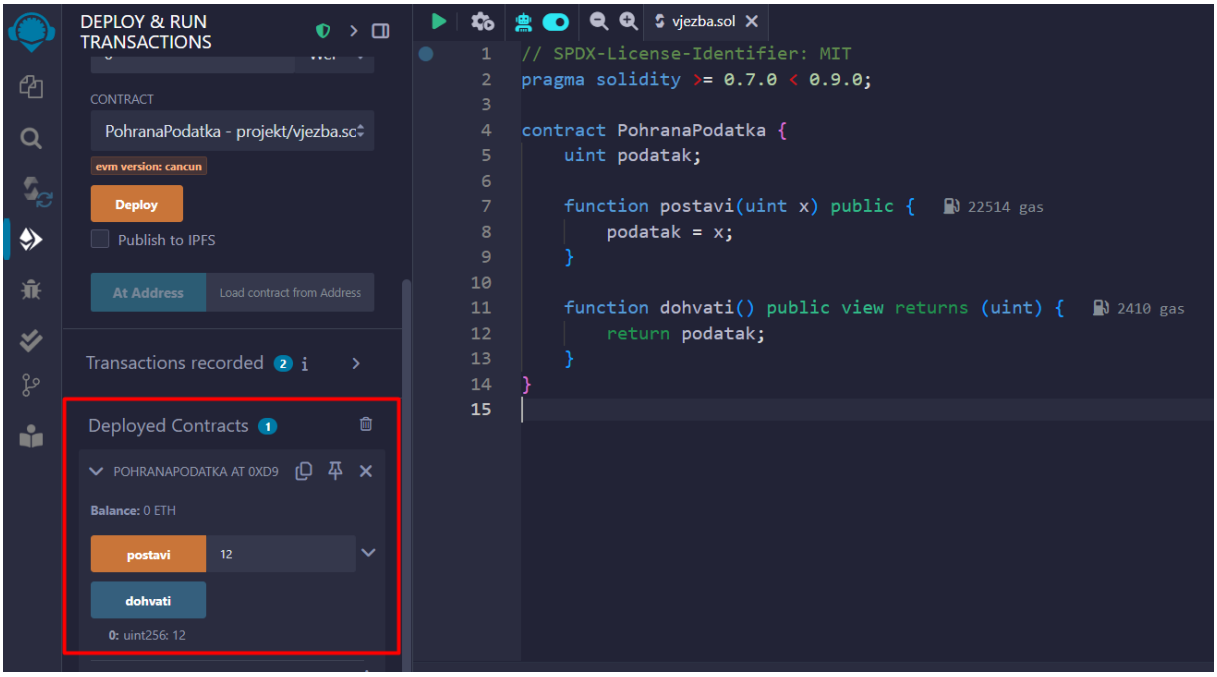
Kreiranje pametnih ugovora u Solidityu slijedi jednostavnu strukturu pisanja koda. Na početku se navodi verzija Soliditya koja se koristi, zatim se definira pametni ugovor s ključnom riječi contract (ugovor) te se unutar ugovora navode razne varijable stanja i funkcije. Varijable sadrže pohranjene podatke unutar ugovora, dok funkcije služe za ostvarivanje interakcija s ugovorom, poput dohvaćanja podatka, mijenjanje vrijednosti, izvršavanje operacija i slično.

```
vjezba.sol
1  // SPDX-License-Identifier: MIT
2  pragma solidity >= 0.7.0 < 0.9.0;
3
4  contract PohranaPodatka {
5      uint podatak;
6
7      function postavi(uint x) public {    22514 gas
8          podatak = x;
9      }
10
11     function dohvati() public view returns (uint) {    2410 gas
12         return podatak;
13     }
14 }
```

Slika 2: Solidity - Primjer strukture programa

Kao što je vidljivo na slici iznad [Slika 2], napravljen je jednostavni pametni ugovor koji omogućuje pohranu i dohvaćanje podatka. Na vrhu se navodi licenca i verziju Soliditya koju ugovor koristi. Ovo omogućuje kompatibilnost sa željenim verzijama kompajlera. Zatim se u nastavku definira pametni ugovor s nazivom PohranaPodatka, te se u njemu navodi varijabla

podatak, koja služi za pohranu brojčane vrijednosti. Funkcija postavi omogućuje postavljanje vrijednosti podatka, dok funkcija dohvati dohvaća zadnje zapisani podatak. Funkcije su javnog tipa (public) pa prema tome svatko može pozvati ove funkcije. Još je bitno istaknuti kako se pored naziva funkcija pojavljuje oznaka gas s brojčanom vrijednošću. Gas je ustvari gorivo koje pokreće transakcije i pametne ugovore. Svaka operacija u Solidityju (npr. dodavanje brojeva, spremanje podataka i druge) troši određenu količinu gasa, a korisnici plaćaju tu potrošnju u Etheru. Optimizacijom koda se može smanjiti potrošnja gasa, čime korisnici plaćaju manje naknade za provođenje transakcija pametnog ugovora. Nakon što je program kompajliran, moguće ga je objaviti i pokrenuti. Jednom objavljeni pametni ugovor postaje trajni dio blockchaina te se nakon tog događaja programski kod ugovora više ne može promijeniti. Iz tog razloga potrebno je dobro osmisliti i testirati pametni ugovor prije nego što postane dostupan svima.



Slika 3: Solidity - Primjer pokretanja programa

Na slici [Slika 3] je prikazan primjer objave i pokretanja pametnog ugovora. Prvo je unesena vrijednost 12, koja je pohranjena u ugovor, a zatim je ta ista vrijednost dohvaćena pomoću javno dostupne funkcije dohvati.

5. Opis slučaja korištenja

Dizajn pametnog ugovora temelji se na osiguravanju transparentnog i sigurnog upravljanja zajedničkim sredstvima pomoću višestrukog potpisa. Njegove glavne funkcionalnosti omogućuju dodavanje ovlaštenih vlasnika, kreiranje transakcija, kontrolirano odobravanje i izvršavanje transakcija. Također, potrebno je implementirati sigurnosne mehanizme koji štite od neovlaštenog pristupa i potencijalnih zloupotreba.

Prilikom inicijalizacije pametnog ugovora, definiraju se vlasnici novčanika, odnosno osobe koje imaju ovlasti za odobravanje i izvršavanje transakcija. Svaki vlasnik se dodaje u sustav prilikom implementacije ugovora, čime se osigurava da samo ovlašteni korisnici mogu upravljati sredstvima unutar ugovora. Jedna od ključnih funkcionalnosti ovog sustava je kreiranje transakcija koje zahtijevaju višestruki potpis. Kada jedan od vlasnika predloži transakciju, ona ne može biti odmah izvršena, već mora prikupiti određeni broj odobrenja od ostalih vlasnika. Ovaj mehanizam osigurava da se sredstva ne mogu trošiti odlukom jednog pojedinca, već je potrebno kolektivno donošenje odluka. Nakon što je transakcija predložena, svaki vlasnik može dati svoj potpis kao potvrdu odobrenja. Kada se postigne unaprijed definirani broj potpisa, transakcija se automatski izvršava. Kako bi se osigurala sigurnost i spriječile zloupotrebe, ugovor implementira niz zaštitnih mehanizama. Potrebno je osigurati da samo vlasnici mogu predlagati i odobravati transakcije, čime se sprječava neovlašteni pristup. Također, ugovor onemogućuje dvostruko potpisivanje iste transakcije od strane istog vlasnika te ne dopušta izvršavanje transakcija koje nisu prikupile potreban broj potpisa. Dodatno, transakcija se ne može ponovno izvršiti nakon što je već odobrena i provedena, čime se izbjegava mogućnost dvostrukog izvršavanja iste transakcije.

Ovaj pametni ugovor dizajniran je tako da osigura sigurno i transparentno upravljanje zajedničkim sredstvima. Nijedna transakcija ne može biti izvršena bez odobrenja više vlasnika. Zahvaljujući višestrukom potpisu i sigurnosnim mjerama, rizici su svedeni na minimum, a samim time se omogućuje sigurno poslovanje unutar danog sustava.

6. Implementacija pametnog ugovora

Višestruko potpisivanje novčanika je implementirano korištenjem Solidity jezika, a temelji se na principu pametnog ugovora. U nastavku će biti prikazani dijelovi koda s objašnjenjima.

6.1. Inicijalizacija varijabli

```
// SPDX-License-Identifier: MIT
pragma solidity >= 0.7.0 < 0.9.0;

contract DigitalniNovcanik {
    address[] public vlasnici;
    mapping(address => bool) public jeVlasnik;
    uint public potrebniPotpisi;

    struct Transakcija {
        address naAdresa;
        uint iznos;
        uint odobrenja;
        bool izvrsena;
    }
```

Slika 4: Programski kod - Inicijalizacija varijabli (1/2)

Na početku programa se navodi da je kod pod MIT licencom, te da podržava verzije Solidity-a između 0.7.0 i 0.9.0. S ključnom riječi *contract* se definira početak pametnog ugovora koji je nazvan *DigitalniNovcanik*. Nadalje, definiraju se varijable koje pohranjuju listu vlasnika, varijabla koja provjera je li određena adresa vlasnik pametnog ugovora te varijabla koja sadrži broj potrebnih potpisa. Varijabla vlasnici je niz adresa te se zato koristi polje adresa. Provjera je li određena adresa vlasnik izvršava se pomoću naredbe *mapping* koja mapira vrijednost adrese na bool. Dakle, ako je vrijednost *true*, onda je navedena adresa, adresa vlasnika. Varijabla *potrebniPotpisi* definira broj potpisa koji je potreban da bi se transakcija mogla izvršiti. *Transakcija* je struktura koja sadrži informacije o transakciji i to adresu na koju se šalje novac (*naAdresa*), iznos koji se šalje (*iznos*), broj vlasnika koji su odobrili transakciju (*odobrenja*) i varijabla koja označava je li transakcija već izvršena (*izvrsena*).

```
    Transakcija[] public transakcije;
    mapping(uint => mapping(address => bool)) public odobreno;
```

Slika 5: Programski kod - Inicijalizacija varijabli (2/2)

Varijabla *transakcije* je niz svih transakcija koje su predložene. Varijabla *odobreno* je mapping koji prati koje su adrese odobrile određenu transakciju. Za svaki indeks transakcije, koristi se mapping adresa koja pokazuje je li ta adresa odobrila transakciju. Ova struktura pomaže u praćenju tko je odobrio koju transakciju.

6.2. Konstruktor

```
constructor(address[] memory _vlasnici, uint _potrebniPotpisi) {    infinite gas 730000 gas
    require(_vlasnici.length > 0, "Mora postojati barem jedan vlasnik");
    require(_potrebniPotpisi > 0 && _potrebniPotpisi <= _vlasnici.length, "Neispravan broj potpisa");

    for (uint i = 0; i < _vlasnici.length; i++) {
        require(!jeVlasnik[_vlasnici[i]], "Vlasnik vec postoji");
        jeVlasnik[_vlasnici[i]] = true;
        vlasnici.push(_vlasnici[i]);
    }

    potrebniPotpisi = _potrebniPotpisi;
}
```

Slika 6: Programski kod - Prikaz konstruktora

Konstruktor je posebna funkcija koja se izvršava samo jednom, i to prilikom implementacije pametnog ugovora. Njegova glavna svrha je inicijalizacija početnih vrijednosti ugovora. U ovom slučaju, konstruktor provjerava postoji li barem jedan vlasnik pametnog ugovora i provjerava je li broj potrebnih potpisa valjan (mora biti između 1 i broja vlasnika). For petlja se koristi za dodavanje adresa vlasnika u niz *vlasnici* i označava odabrane adrese kao vlasnike u mappingu varijable *jeVlasnik*. Na ovaj način se osigurava da je ugovor pravilno definiran što uključuje postavljanje vlasnika i pravila odobravanja transakcije.

6.3. Predlaganje transakcije

```
function predajTransakciju(address _naAdresa, uint _iznos) public {    103020 gas
    require(jeVlasnik[msg.sender], "Samo vlasnici mogu predloziti transakciju");

    transakcije.push(Transakcija({
        naAdresa: _naAdresa,
        iznos: _iznos,
        odobrenja: 0,
        izvrsena: false
    }));
}
```

Slika 7: Programski kod - Funkcija predlaganje transakcije

Funkcija *predajTransakciju* omogućuje vlasnicima da predlože nove transakciju. Pri početku funkcije se provjerava je li osoba koja traži provedbu transakcije uistinu vlasnik digitalnog novčanika. *Msg.sender* je adresa korisnika koji poziva funkciju te ukoliko je ona lažna (korisnik nije vlasnik digitalnog novčanika) onda se cijela funkcija prekida i vraća poruka greške. Ukoliko se utvrdi da je osoba vlasnik, tada se kreira novi objekt tipa *Transakcija* koji sadrži podatke o primatelju, iznos, broj odobrenja te status transakcije. Jednom kreirani objekt *Transakcija* se dodaje u niz transakcija putem naredbe *push*.

6.4. Odobravanje transakcije

```
function odobriTransakciju(uint _txIndex) public {    infinite gas
    require(jeVlasnik[msg.sender], "Samo vlasnici mogu odobriti");
    require(_txIndex < transakcije.length, "Neispravan indeks");
    require(!transakcije[_txIndex].izvrsena, "Transakcija je vec izvrsena");
    require(!odobreno[_txIndex][msg.sender], "Vec ste odobrili");

    odobreno[_txIndex][msg.sender] = true;
    transakcije[_txIndex].odobrenja++;

    if (transakcije[_txIndex].odobrenja >= potrebniPotpisi) {
        izvrsiTransakciju(_txIndex);
    }
}
```

Slika 8: Programski kod - Funkcija odobravanja transakcije

Odobravanje određene transakcije se provodi putem funkcije *odobriTransakciju*. Na početku se provjerava je li osoba koja želi odobriti transakciju vlasnik te je li unesen valjan broj indeksa transakcije. Ako se pokuša odobriti nepostojeća transakcija, funkcija se prekida s porukom greške. Nadalje, provjerava se je li već transakcija izvršena, te se time sprječava višestruko izvršavanje iste transakcije. Zadnja provjera koja se odvija je provjera je li vlasnik već odobrio određenu transakciju. Ova provjera prati adrese koje su već dale svoj potpis, te sprječava mogućnost duplog davanja potpisa iste transakcije. Ako su sve provjere uspješno prošle, korisnikov potpis se pohranjuje. Varijabla *odobreno* označava da je korisnik odobrio ovu transakciju, te se povećava broj odobrenja za ovu transakciju. U slučaju da je broj prikupljenih potpisa jednak ili veći od definiranog, poziva se funkcija *izvrsiTransakciju*, koja zapravo provodi transakciju. Ovo omogućuje automatsko izvršavanje transakcije čim se postigne potreban broj potpisa.

6.5. Izvršenje transakcije

```
function izvrsiTransakciju(uint _txIndex) internal {    infinite gas
    Transakcija storage txn = transakcije[_txIndex];
    require(txn.odobrenja >= potrebniPotpisi, "Nema dovoljno potpisa");
    require(!txn.izvrsena, "Vec izvrseno");

    txn.izvrsena = true;
    payable(txn.naAdresa).transfer(txn.iznos);
}

receive() external payable {}    undefined gas
}
```

Slika 9: Programski kod - Funkcija izvršenja transakcije

Ova funkcija se poziva iz funkcije *odobriTransakciju*, a izvršava se nakon što je prikupljen dovoljan broj potpisa. Također, ovo je prva funkcija koja nije javno dostupna

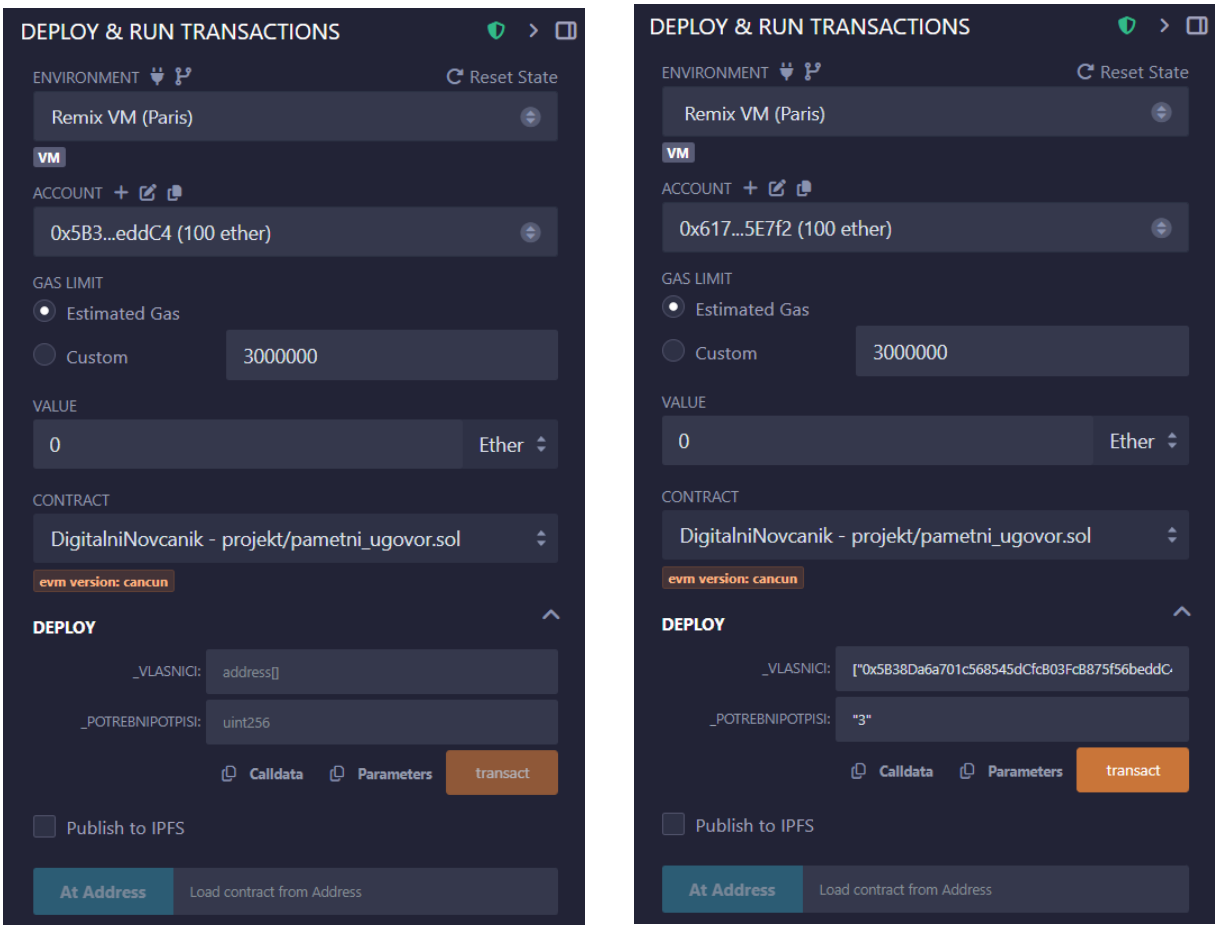
(*internal*), već ju mogu pozvati samo druge funkcije unutar pametnog ugovora (ne može ju direktno pozvati korisnik izvana). *Txn* pokazuje na transakciju na indeksu u nizu transakcije. Koristi se *storage* što znači da se *txn* referencira direktno na podatke iz blockchaina, to jest da se podatci ne kopiraju. Prije nego što se transakcija izvede, funkcija provjerava dva uvjeta i to je li broj odobrenja veći ili jednak potrebnom broju potpisa, te je li transakcija već izvršena. Ukoliko provjere prođu, transakcija se postavlja na vrijednost izvršena, što sprječava duplo izvršavanje iste transakcije. Predzadnja linija koda šalje novac primatelju, gdje je navedena adresa primatelja, te iznos koji se šalje. Ako transakcija uspije, sredstva se prebacuju s računa na račun. Postoji i mogućnost da se transakcija ne uspije izvršiti jer vlasnici digitalnog novčanika nemaju dovoljno sredstava na računu (Ethera). U tom slučaju transakcija će se prekinuti s greškom. Zadnja linija koda omogućuje da digitalni novčanik može primiti sredstva. Ova funkcija je potrebna kako bi novčanik mogao prihvatiti Ether od drugih korisnika.

7. Testiranje pametnog ugovora

Za testiranje pametnog ugovora, prvo ga je potrebno pokrenuti odnosno deployati. Nakon prikaza pokretanja digitalnog novčanika s odgovarajućim unosima, izvršit će se transakcija te objasniti rad svih funkcija na primjeru. Na posljetku, preostaje simulirati ljudske pogreške i lažne unose, kako bi se uvjerili da je pametni ugovor potpuno siguran.

7.1. Pokretanje digitalnog novčanika (Deploy)

Jednom kada je programski kod digitalnog novčanika napisan, potrebno ga je kompajlirati, te nakon slijedi deployanje. Na sljedećoj slici [Slika 10] je prikazan prazan zaslon za pokretanje digitalnog novčanika te ispunjeni zaslon s odgovarajućim unosima.



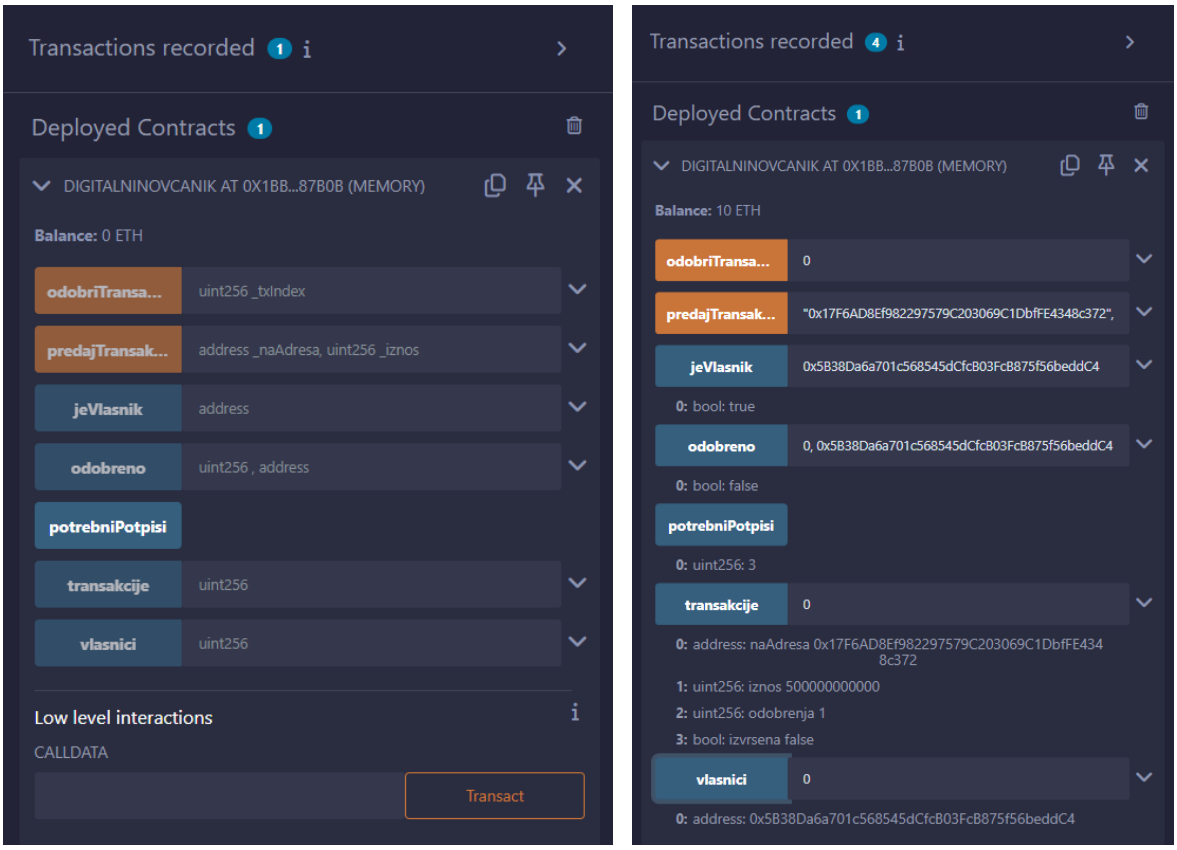
Slika 10: Pokretanje pametnog ugovora - Prazan zaslon (lijevi) i ispunjeni (desni)

Opcija *Environment* sadrži različita virtualna okruženja na kojima se odvija Etherium mreža. Ovdje je odabrana Remix virtualna mašina (Paris), koja nudi besplatno okruženje te je ta opcija zadana vrijednost (eng. *default*). Odjeljak *Account* sadrži profile, opisane pomoću dugog niza znakova koji predstavljaju virtualnu adresu određenog profila. Svaki od tih profila

ima račun s iznosom od 100 Ethera. Koriste se za postavljanje vlasnika nad pametnim ugovorom te za izvođenje funkcija i transakcija. Za svaku transakciju i izvršavanje funkcije se troši određena količina Ethera kojom svaki profil raspolaže. Sekcija *Gas Limit* nije korištena za provođenje digitalnog novčanika te su ostavljane zadane vrijednosti. Opcija *Value* služi za prenošenje Ethera s korisničkog profila na pametni ugovor, ili u ovom slučaju na digitalni novčanik. Na kraju slijedi najbitniji dio ugovora, gdje se unose vrijednosti za pokretanje pametnog ugovora. Varijabla vlasnici očekuje unos adresa svih osoba koje su vlasnici digitalnog novčanika, dok varijablom potrebni potpisi se definira koliko vlasnika mora odobriti određenu transakciju prije nego što se izvrši. U primjeru je uneseno pet različitih adresa profila, te su potrebna tri potpisa za provođenje transakcije. Klikom na opciju *transact* se pokreće ugovor, te kao takav postaje nepromjenjiv na Etherum mreži.

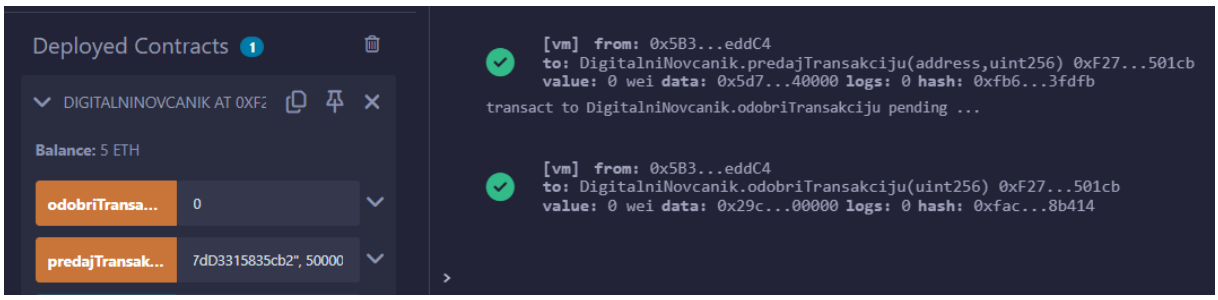
7.2. Provedba transakcije

Pokrenutom ugovoru moguće je pristupiti putem opcije *Deployed Contracts*. Ovdje se nalazi pristup svim funkcijama, te se također nalazi prostor za unos vrijednosti varijabli.



Slika 11: Provedba transakcije – Prazan zaslon (lijevi) i ispunjeni (desni)

Na slici iznad [Slika 11], s lijeve strane prikazan je upravo pokrenut pametni ugovor, bez ijedne unesene vrijednosti, dok je s desne strane prikazan isti taj ugovor ispunjen s različitim vrijednostima. Postoje dva dijela pametnog ugovora, dio koji bilježi provedbu svake transakcije i dio koji služi za interakciju s ugovorom. Prilikom interakcije s ugovorom zabilježene su četiri transakcije koje je moguće ispisati u JSON formatu. Kreirani ugovor sadrži dva polja koja služe za unos vrijednosti u ugovor (označena narančastom bojom) i pet opcija koje služe za dohvaćanje vrijednosti i stanja iz pametnog ugovora, odnosno digitalnog novčanika. Prvo je potrebno kreirati transakciju. U polje se unosi adresa na koju se šalje novac te iznos novca izraženog u Weima (1 Ether ima 1 trilijun Weia – 10^{18} Wei). Klikom na *predajTransakciju* transakcija se predaje te se bilježi u popis provedenih transakcija. Za odobriti transakciju korisnik mora unijeti indeks (redni broj) transakcije koju želi odobriti. S obzirom da je ova transakcija prva u nizu korisnik unosi vrijednost 0. Klikom na *odobriTransakciju* transakcija će se odobriti ukoliko je osoba vlasnik tog ugovora. U slučaju da osoba nije vlasnik program će vratiti pogrešku. Funkcija *jeVlasnik* služi za provjere određene adrese je li ona vlasnik digitalnog novčanika ili nije. Ova funkcija vraća bool vrijednosti istina (*true*) ili laž (*false*). Nadalje, pomoću opcije odobreno može se provjeriti je li osoba odobrila transakciju ili nije. Za unos se uzima indeks transakcije i adresa osobe koju provjeravamo. Opcija potrebni potpisi vraća vrijednost koliko je potrebno potpisa da se transakcija odobri. Ova vrijednost je definirana pokretanjem ugovora, a potrebno je tri potpisa. Funkcija *transakcije* omogućuje pregled stanja određene transakcije. Unosi se redni broj transakcije, a funkcija vraća informacije o tome na koje se adresu novac šalje, koliki je iznos koji se šalje, koliko transakcija ima odobrenih potpisa te je li transakcija izvršena ili nije. Zadnja opcija koja je na raspolaganju korisnicima digitalnog novčanika je opcija vlasnici koja navodi adresu vlasnika digitalnog novčanika. Prilikom unosa potrebno je unijeti redni broj vlasnika.



Slika 12: Uspješno izvršena transakcija

Jednom kada se skupi potrebni broj potpisa, transakcija se automatski izvršava. Time se oduzimaju novčana sredstva s digitalnog novčanika i šalju se na postavljenu adresu korisnika. Na priloženoj slici [Slika 12] se može vidjeti da je trenutačni iznos digitalnog novčanika 5 ETH, a da je na početku bio 10 ETH.

7.3. Simulacija pogrešaka i lažnih unosa

Simulacija pogrešaka i lažnih unosa ima za cilj prikazati provjere koje se događaju unutar pametnog ugovora, koje osiguravaju da se pametni ugovor izvodi onako kako je i zamišljen. U prvom slučaju je simulirana opcija gdje vlasnik novčanika pokušava odobriti transakciju koja je već provedena i izvršena. Sustav mu vraća pogrešku koja je označena na slici s crvenom bojom te se navodi poruke greške „Transakcija je već izvršena“. U sljedećem slučaju, osoba koja nije vlasnik digitalnog novčanika pokušava odobriti transakciju. Ovdje sustav provjerava je li trenutačna osoba vlasnika, i ako nije, vraća porugu greške „Samo vlasnici mogu odobriti“, što je prikazano sljedećom slikom [Slika 13].

```
[vm] from: 0x5B3...eddC4 to: DigitalniNovcanik.odobriTransakciju(uint256) 0x1bB...87B0b value: 0 wei data: 0x29c...00000
logs: 0 hash: 0xa82...3e714
transact to DigitalniNovcanik.odobriTransakciju errored: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Transakcija je vec izvrsena".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

transact to DigitalniNovcanik.odobriTransakciju pending ...

[vm] from: 0x5c6...21678 to: DigitalniNovcanik.odobriTransakciju(uint256) 0x1bB...87B0b value: 0 wei data: 0x29c...00003
logs: 0 hash: 0xb2e...43534
transact to DigitalniNovcanik.odobriTransakciju errored: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Samo vlasnici mogu odobriti".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

transact to DigitalniNovcanik.predajTransakciju pending ...
```

Slika 13: Simulacija pogrešnih unosa (1/2)

Treći slučaj prikazuje da samo vlasnik digitalnog novčanika može predložit slanje transakcije. Dakle, osoba koja nije vlasnik nema pravo predlaganja ni sudjelovanja u provođenju transakcija. Zadnja simulirana greška prikazuje korisnika koji pokušava pristupiti podacima transakcije neispravnog indeksa. Neispravan indeks transakcije je u ovom slučaju broj transakcije koja ne postoji, primjerice predložene su tri transakcije, a korisnik pokušava pristupiti transakciji na indeksu pet.

```
[vm] from: 0xCA3...a733c to: DigitalniNovcanik.predajTransakciju(address,uint256) 0x1bB...87B0b value: 0 wei
data: 0x5d7...28800 logs: 0 hash: 0x4b5...230d0
transact to DigitalniNovcanik.predajTransakciju errored: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Samo vlasnici mogu predloziti transakciju".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

transact to DigitalniNovcanik.odobriTransakciju pending ...

[vm] from: 0xAb8...35cb2 to: DigitalniNovcanik.odobriTransakciju(uint256) 0x1bB...87B0b value: 0 wei data: 0x29c...00003
logs: 0 hash: 0x4be...e201e
transact to DigitalniNovcanik.odobriTransakciju errored: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Neispravan indeks".
If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

Slika 14: Simulacija pogrešnih unosa (2/2)

8. Zaključak

Ideja višestrukog potpisivanja je korisna metoda za povećanje sigurnosti u upravljanju digitalnim novcem. Umjesto da jedna osoba ima potpunu kontrolu nad novčanim sredstvima, ovakav sustav zahtijeva odobrenje više korisnika prije nego što se neka transakcija može provesti. To pomaže u sprečavanju zloupotrebe, pogrešaka i krađe, jer se sprječava samostalno donošenje odluka bez dogovora s drugima.

Rad prikazuje što je višestruko potpisivanje i zašto je važno, te ističe glavne prednosti korištenja višestrukog potpisivanja novčanika. Zatim su objašnjeni osnovni koncepti razvojne okoline u kojoj je izrađen primjer pametni ugovor koji implementira značajke višestrukog potpisivanja novčanika. Nakon teorijskog definiranja dizajna pametnog ugovora, proveden je praktični dio, gdje je izrađen pametni ugovor u programskom jeziku Solidity. Ugovor omogućuje dodavanje, odobravanje i izvršavanje transakcija uz dogovor više vlasnika. Korišten je online dostupan alat Remix - Ethereum IDE. Za izradu programskog rješenja korištene su varijable polja i numeričkih vrijednosti, a funkcije uz normalno izvođenje nude niz zaštitnih mjera koje pružaju nesmetani rad pametnog ugovora. Napisani kod provjeren je kroz testiranje normalnog izvođenja digitalnog novčanika te je naposljetku provjereno kako se digitalni novčanik ponaša kod pogrešaka bilo slučajnih ili namjernih.

Sve u svemu, pametni ugovor pruža jednu potpuno funkcionalnu cjelinu koji donosi visoku dozu sigurnosti i transparentnosti. To je posebno važno u današnjem svijetu, gdje su prisutne kriptovalute i mnogobrojne digitalne inovacije. Danas se većina sredstva i resursa nalazi u digitalnom obliku i prema tome potrebno se je prilagoditi zahtjevima digitalnog doba koje iziskuje nove digitalne metode i rješenja.

Popis literature

[1] *Lekcija 1: Osnove Blockchaina - Kriptomat*. (bez dat.). Pristupano 13. travnja, 2025, sa https://kriptomat.io/hr/nauci/akademija/osnove-blockchaina/osnove-blockchaina/

[2] *Lekcija 3: Sve o pametnim ugovorima - Kriptomat*. (bez dat.). Pristupano 13. travnja, 2025, sa https://kriptomat.io/hr/nauci/akademija/osnove-blockchaina/sve-o-pametnim-ugovorima/

[3] Goel, A. K., Bisht, V. S., & Chaudhary, S. (2023). Multisignature Crypto Wallet Paper. *Proceedings of the 8th International Conference on Communication and Electronics Systems, ICCES 2023*, 476–479. https://doi.org/10.1109/ICCES57224.2023.10192591

[4] Ibrahimpašić, B. I., & Lidan, E. (2010). Digitalni potpis. *Osječki Matematički List*, *10*, 139–148.

[5] I. Šimić, "Bitcoin kao oblik digitalne valute i elektroničkog plaćanja", Specijalistički diplomski stručni, Libertas međunarodno sveučilište, Zagreb, 2017. Dostupno na: https://urn.nsk.hr/urn:nbn:hr:223:055494

[6] D. Brocić, "Digitalni novčanik kao alat u digitalnim transakcijama", Završni rad, Sveučilište Josipa Jurja Strossmayera u Osijeku, Ekonomski fakultet u Osijeku, Osijek, 2018. Dostupno na: https://urn.nsk.hr/urn:nbn:hr:145:571530

[7] *Remix - Ethereum IDE & community*. (bez dat.). Pristupano 29. ožujka, 2025, sa https://remix-project.org/?lang=en

[8] *Remix - Ethereum IDE*. (bez dat.) Pristupano 29. ožujka, 2025, sa https://remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.26+commit.8a97fa7a.js

[9] *About | Solidity Programming Language*. (bez dat.) Pristupano 29. ožujka, 2025, sa https://soliditylang.org/about/

Popis slika

Slika 1: Remix Online IDE - Početni zaslon 6
Slika 2: Solidity - Primjer strukture programa 7
Slika 3: Solidity - Primjer pokretanja programa 8
Slika 4: Programski kod - Inicijalizacija varijabli (1/2) 10
Slika 5: Programski kod - Inicijalizacija varijabli (2/2) 10
Slika 6: Programski kod - Prikaz konstruktora 11
Slika 7: Programski kod - Funkcija predlaganje transakcije 11
Slika 8: Programski kod - Funkcija odobravanja transakcije 12
Slika 9: Programski kod - Funkcija izvršenja transakcije 12
Slika 10: Pokretanje pametnog ugovora - Prazan zaslon (lijevi) i ispunjeni (desni) 14
Slika 11: Provedba transakcije – Prazan zaslon (lijevi) i ispunjeni (desni) 15
Slika 12: Uspješno izvršena transakcija 16
Slika 13: Simulacija pogrešnih unosa (1/2) 17
Slika 14: Simulacija pogrešnih unosa (2/2) 17